Практическая работа

Отчёт по практической работе № 7

Реализация безопасной аутентификации

Дисциплина:	Основы разработки защищённого программного обеспечения и компьютерных сетей
Вариант:	19 — Система учёта транспортных средств организации
Исходная версия:	v0.6
Версия работы:	v0.7
Студент:	Мостовая Полина Андреевна
Группа:	ИУСбд-02-23

2026

Содержание

1 Тема, цель и характеристика среды 2

2 Вариант 19 и границы реализации 2

3 Архитектура аутентификации 2

4 Хранение паролей 3

5 Ограничение попыток и вариант 19 3

6 Управление сеансами 3

7 Восстановление доступа 4

8 Фрагменты исходного кода 4

9 Результаты тестирования 4

10 Ручные скриншоты 5

11 Остаточные риски 8

12 Вывод 8

13 Ответы на контрольные вопросы 8

Список использованных источников 9

1. Тема, цель и характеристика среды

Тема работы — безопасная аутентификация в сквозном проекте учёта транспортных средств. Цель — реализовать регистрацию, парольную проверку, временную блокировку, серверные сеансы и одноразовое восстановление, затем проверить их автоматизированными сценариями. Работа продолжает v0.6, сохраняя её модули учёта обращений, файлового хранилища, RBAC и валидации.

Проверенная среда: macOS, Python 3.12.14 из виртуального окружения, pytest 9.0.2, SQLite из стандартной библиотеки, XeTeX (TeX Live 2026). Проект не имеет веб-интерфейса или сетевого API.

2. Вариант 19 и границы реализации

Индивидуальное задание — анализ риска блокировки чужой учётной записи. Нарушитель знает логин сотрудника, но не пароль; пять неверных попыток вызывают временный отказ владельцу при создании нового сеанса. Это злоупотребление средством защиты доступности. Brute Force проверяет много кандидатов для одной записи; Password Spraying — небольшое число типовых паролей для многих записей; вариант 19 не пытается угадать пароль.

AuthService подтверждает только серверный числовой `user_id`. Он не назначает роль из пользовательского ввода и пока не подключён к сервисам автопарка. Будущая интеграция должна получать роль из серверного отображения подтверждённого id к RBAC. Это не веб-сервер и не MFA.

3. Архитектура аутентификации

SQLite предоставляет локальное хранилище, а ограниченные параметры запросов и транзакции используются стандартным интерфейсом Python. [5] Вход начинается с недоверенных строк логина и пароля. После проверок SQLite возвращает соль, стоимость KDF и проверочное значение. PBKDF2 вычисляет кандидат, сравнение выполняется `hmac.compare_digest`. Успех создаёт токен сеанса, но в БД записывается лишь SHA-256. При ошибке обновляется счётчик. Редактируемая схема: `docs/auth_registration_login.mmd`.

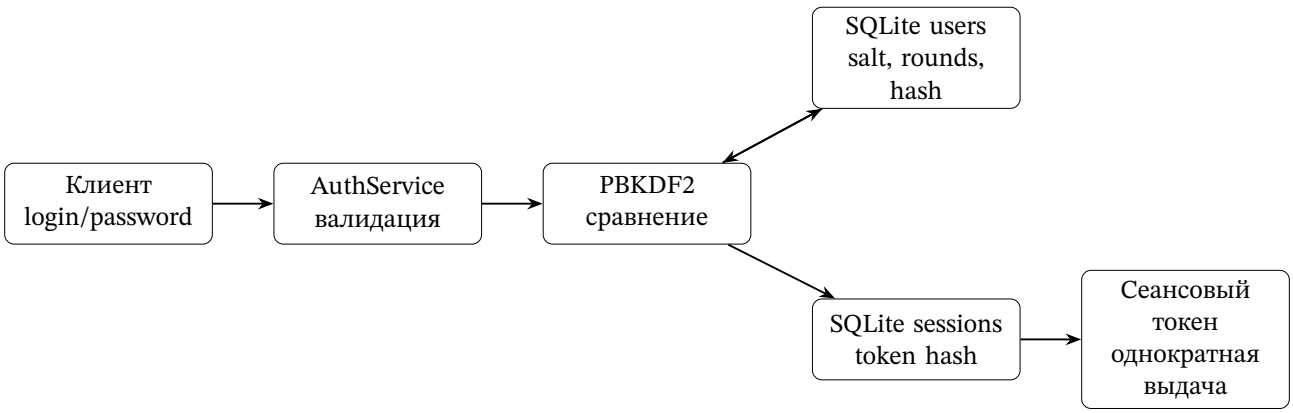


Рис. 1: Проверка пароля и создание серверного сеанса.

4. Хранение паролей

NFC-нормализация выполняется средствами стандартного модуля Unicode. [6] Принципы применения PBKDF2 и соли сверены с документацией Python и OWASP. [2, 7] Логин принимается только как `str`, нормализуется `casefold()`, затем проверяется шаблон `[a-z][a-z0-9_]2,31`. Новый пароль нормализуется NFC; допустимая длина после нормализации — 15–128 символов, максимальный размер UTF-8 — 1024 байта. Пробелы не удаляются, регистр не меняется. Запрещающий список содержит лишь три учебных примера и не покрывает известные утечки.

Для каждого пароля используются 16 случайных байтов соли и PBKDF2-HMAC-SHA-256 с 600 000 итераций и результатом 32 байта. В БД записывается и число итераций. SHA-256 слишком быстр для пароля с низкой энтропией; для случайного высокоэнтропийного токена быстрый SHA-256 используется как проверочное значение.

5. Ограничение попыток и вариант 19

Общий отказ при ошибке аутентификации рекомендован для уменьшения перечисления пользователей; счётчики попыток также требуют анализа риска блокировки. [8, 11] После пяти последовательных ошибок блокировка действует 60 секунд. Правильный пароль до срока также не даёт новый сеанс. При точном равенстве времени границе блокировка окончена; успех сбрасывает счётчик. Неизвестный логин вызывает фиктивное вычисление KDF, которое уменьшает очевидное временное различие, но не исключает все тайминг-каналы.

Исследование использовало только временную локальную БД и искусственную запись с управляемыми часами. Проверены штатный вход владельца, пять неудач от другого участника, невозможность создать новый сеанс в течение блокировки, восстановление входа на границе 60 секунд и независимость другой учётной записи. Уже выданный сеанс этой блокировкой не отзывается. Ограничение затрудняет часть онлайн-подбора, но может использоваться для отказа в обслуживании конкретному владельцу.

Возможные будущие меры: ограничения источника и общей частоты, нарастающая задержка, сочетание нескольких сигналов, уведомление, challenge подозрительных попыток, контроль массовых обращений и безопасное восстановление доступности. Они предложены, но не реализованы. Фиксированные 60 секунд не являются универсальной защитой от распределённого подбора.

6. Управление сеансами

Случайная генерация токенов и серверное хранение их состояния соответствуют общим рекомендациям по сессиям. [4, 9] Токен создаётся через `secrets.token_urlsafe(32)`. Сервер хранит SHA-256, user id, время создания и последнюю активность. `resolve()` проверяет формат, активность учётной записи, абсолютный срок 3600 секунд и срок бездействия 900 секунд; при успехе обновляет `last_seen`. `logout()` удаляет серверную запись. Удаление клиентской копии не отзывает скопированный токен. Схема: `docs/auth_session.mmd`.

7. Восстановление доступа

Одноразовые коды, отзыв сеансов и отдельный последующий вход рассматриваются в материалах по восстановлению. [10, 11] При регистрации генерируется случайный recovery-код, который выдаётся вызывающей стороне однократно; БД хранит его SHA-256. Восстановление проверяет логин, код и новый пароль, заменяет соль, хеш пароля и хеш нового кода в транзакции и отзывает все сеансы. Новый сеанс автоматически не выдаётся; выполняется отдельный обычный вход. Старый код становится недействительным. Это учебная модель предварительно выданного кода, а не процесс подтверждения личности через канал связи. Схема: `docs/auth_recovery.mmd`.

8. Фрагменты исходного кода

Реализация парольной KDF и recovery-проверочного значения:

```
salt = secrets.token_bytes(16)
password_hash = hashlib.pbkdf2_hmac(
    "sha256", password_bytes, salt, self.rounds, dklen=32
)
recovery_hash = hashlib.sha256(recovery_code.encode("ascii")).digest()
```

Проверка пароля и серверная запись токена:

```
candidate = self._derive(password_bytes, salt, rounds)
password_ok = hmac.compare_digest(candidate, stored)
token = secrets.token_urlsafe(32)
digest = self._token_hash(token)
db.execute("INSERT INTO sessions(token_hash,user_id,created,last_seen) VALUES
   (?,?,?,?)",
           (digest, row["id"], now, now))
```

Полный код находится в `app/auth.py`; фрагменты соответствуют версии v0.7.

9. Результаты тестирования

Тестовый экземпляр допускает 1000 итераций только для ускорения изолированных тестов; стандартное значение остаётся 600 000. Полный локальный набор: 149 тестов и 9 subtests пройдены. Набор v0.7 содержит 29 тестов. Отдельный запуск демонстрации фактически выполнил регистрацию, вход и восстановление при 600 000 итерациях.

№	Сценарий	Ожидаемый результат	Факт
1	Правильный пароль	Вход разрешён	Пройдено
2	Неверный пароль	Отказ	Пройдено
3	Повторный логин	Отказ регистрации	Пройдено
4	Равные пароли разных пользователей	Разные соли	Пройдено
5	Проверка хранилища	Пароля в открытом виде нет	Пройдено
6	Короткий пароль	Отклонён	Пройдено
7	Добавочный пробел	Считается частью пароля	Пройдено

№	Сценарий	Ожидаемый результат	Факт
8	Неизвестная запись	Отказ	Пройдено
9	Пять ошибок	Временная блокировка	Пройдено
10	Истечение блокировки	Вход на точной границе	Пройдено
11	Выход	Токен отозван	Пройдено
12	Срок бездействия	Сеанс недействителен	Пройдено
13	Абсолютный срок	Сеанс недействителен	Пройдено
14	Восстановление	Прежние сеансы отозваны	Пройдено
15	Неверный recovery-код	Отказ без изменений	Пройдено
16	Вариант 19: штатный вход	Успех до атаки	Пройдено
17	Вариант 19: пять ошибок постороннего	Владелец временно заблокирован	Пройдено
18	Вариант 19: истёкший срок	Вход восстановлен	Пройдено
19	NFC и длина 15/128/129	Нормализация и границы	Пройдено
20	Размер UTF-8 и слабый пароль	Недопустимое отклонено	Пройдено
21	Регистр логина и типы	Casefold и строгие типы	Пройдено
22	Неверный токен и его хранение	Отказ; в БД только хеш	Пройдено
23	Одноразовость recovery-кода	Повторный код отклонён	Пройдено
24	Изоляция разных логинов	Другой вход доступен	Пройдено
25	Сброс ошибок после успеха	Счётчик обнулён	Пройдено
26	Обновление активности	Срок бездействия продлён	Пройдено
27	Восстановление	Новый сеанс не создаётся	Пройдено
28	Повреждённая соль	Безопасный отказ	Пройдено
29	Уже действующий сеанс при блокировке новых входов	Не отзывается блокировкой	Пройдено

10. Ручные скриншоты

Снимки получены из Terminal.app после запуска соответствующих режимов демонстрации. На них не видны пароли, резервные коды или действующие токены. Каждое изображение соответствует фактическому выводу локальной демонстрации и тестов.

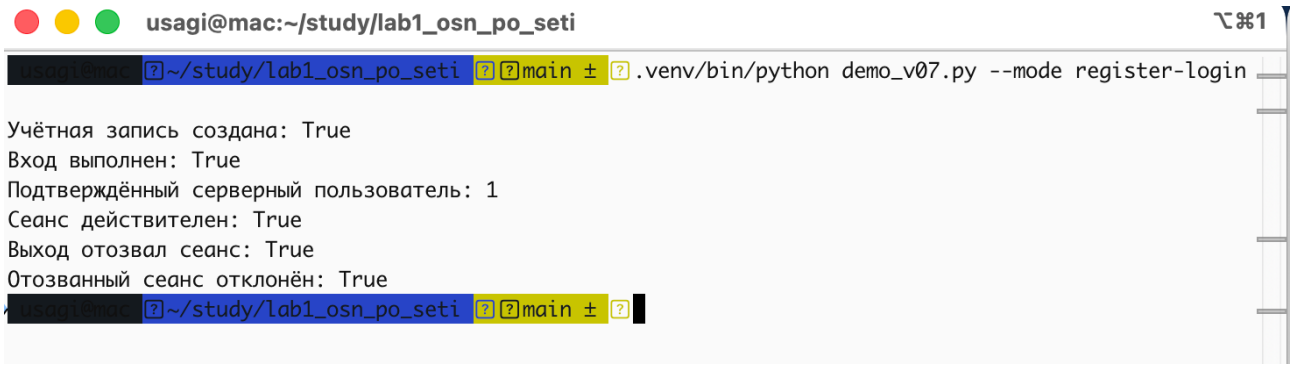


Рис. 2: Регистрация, вход, проверка серверной идентичности и отзыв сеанса.



Рис. 3: Различие солей и парольных проверочных значений при одинаковом пароле.

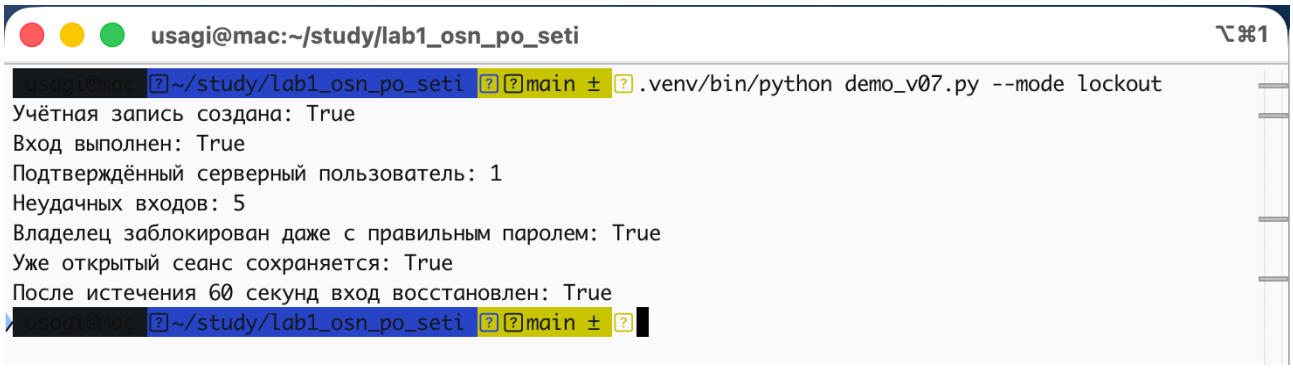


Рис. 4: Пять неудачных попыток, отказ корректному паролю и истечение блокировки.

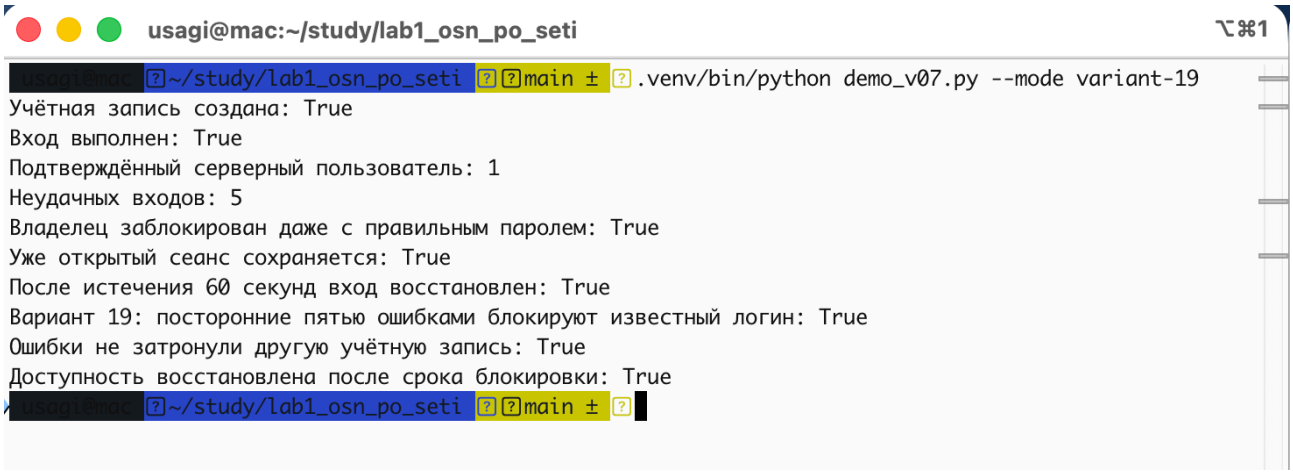


Рис. 5: Вариант 19: блокировка чужого логина и восстановление доступности.

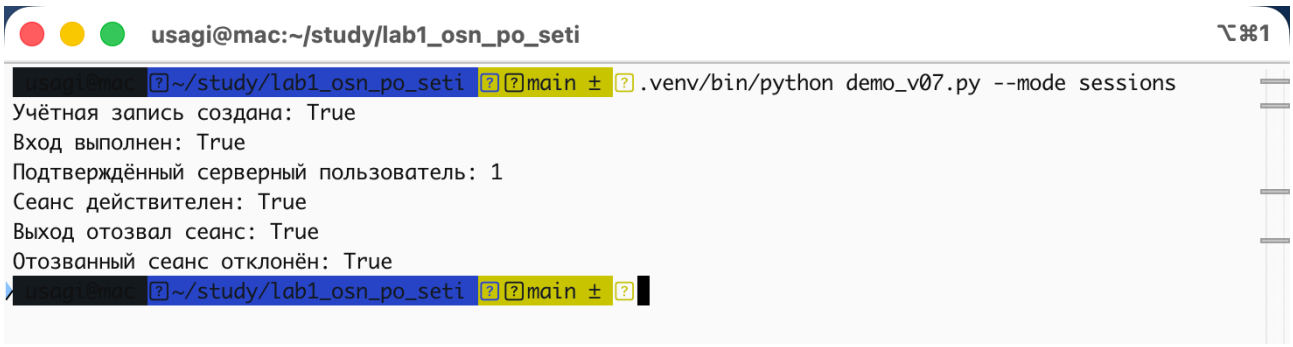


Рис. 6: Проверка действующего сеанса и отказ после серверного отзыва.



Рис. 7: Одноразовое восстановление доступа и отзыв прежнего сеанса.

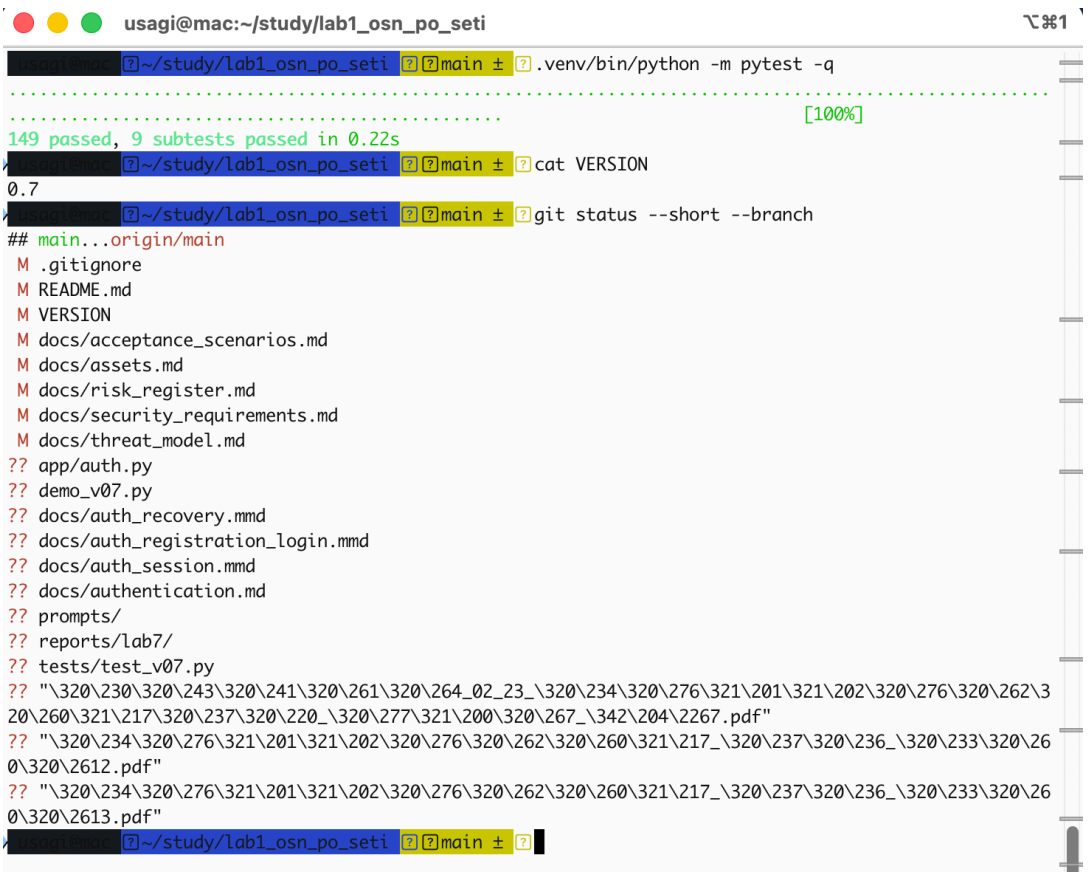


Рис. 8: Фактические результаты регрессионных тестов, VERSION и состояние рабочей копии.

11. Остаточные риски

Учебный список слабых паролей неполон; PBKDF2 не memory-hard; фиктивный KDF не устраняет все временные различия; блокировка создаёт отказ в доступности и не устраняет распределённый перебор. SQLite предназначена здесь для локального процесса и файла. Кража активного токена или recovery-кода остаётся значимой. Отсутствуют MFA, сетевое ограничение частоты, уведомления, подтверждённые каналы восстановления, назначение ролей и связь с бизнес-операциями автопарка.

12. Вывод

Создан локальный AuthService с индивидуальными солями, PBKDF2-HMAC-SHA-256, блокировкой после пяти ошибок, проверкой и отзывом сеансов, одноразовым восстановлением и отзывом прежних сеансов. Пройдены 149 регрессионных тестов и 9 subtests; стандартные 600 000 итераций использованы в отдельной регистрации, входе и восстановлении. Вариант 19 исследован на искусственной локальной записи. Результаты ручных демонстраций подтверждены семью скриншотами, включёнными в отчёт.

13. Ответы на контрольные вопросы

1. Идентификация — указание рассматриваемой учётной записи.
2. Аутентификация проверяет доказательство контроля этой записи.
3. Открытый пароль после утечки сразу раскрывает секрет пользователя.
4. Соль — случайное значение, индивидуализирующее парольную проверку.
5. Разные соли скрывают совпадение проверочных значений равных паролей.
6. SHA-256 слишком быстр для перебора низкоэнтропийных паролей.
7. PBKDF2 — многократное преобразование пароля через HMAC для повышения стоимости проверки.
8. Argon2id использует память и вычисления, затрудняя параллельный массовый перебор.
9. `compare_digest` снижает зависимость времени сравнения от общего префикса значений.
10. Пробел может быть частью секрета; его удаление меняет ввод.
11. NFC приводит канонически эквивалентный Unicode к согласованной форме.
12. Список помогает отклонить известные слабые значения, но малый список неполон.
13. Онлайн-подбор проходит через серверные ограничения; офлайн-подбор работает с утёкшими хешами отдельно от сервера.
14. Постоянная блокировка позволяет постороннему лишить владельца доступа.
15. Серверный сеанс связывает предъявляемый токен с учётной записью и сроками.
16. Непредсказуемость мешает угадыванию действующего токена.
17. Абсолютный срок считается с создания, срок бездействия — с последней активности.
18. Удаление локальной копии не удаляет скопированный токен у злоумышленника.
19. Одноразовый код не должен повторно открывать восстановление после применения.
20. Отзыв не позволяет старому сеансу сохранять доступ после смены пароля.
21. Обычный отдельный вход подтверждает пароль и создаёт сеанс штатным путём.
22. Остаются фишинг, компрометация устройства/БД, слабые пароли, отказ в обслуживании, кража токенов, временные различия и ошибки интеграции полномочий.

Список использованных источников

[1] Методические указания «Практическая работа № 7. Реализация безопасной аутентификации». Предоставленный PDF, 33 с.

[2] Python Software Foundation. `hashlib` — Secure hashes and message digests. URL: https://docs.python.org/3/library/hashlib.html (дата обращения: 29.10.2026).

[3] Python Software Foundation. `hmac` — Keyed-Hashing for Message Authentication. URL: https://docs.python.org/3/library/hmac.html (дата обращения: 29.10.2026).

[4] Python Software Foundation. `secrets` — Generate secure random numbers for managing secrets. URL: https://docs.python.org/3/library/secrets.html (дата обращения: 29.10.2026).

[5] Python Software Foundation. `sqlite3` — DB-API 2.0 interface for SQLite databases. URL: https://docs.python.org/3/library/sqlite3.html (дата обращения: 29.10.2026).

[6] Python Software Foundation. `unicodedata` — Unicode Database. URL: https://docs.python.org/3/library/unicodedata.html (дата обращения: 29.10.2026).

[7] OWASP Foundation. Password Storage Cheat Sheet. URL: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html (дата обращения: 29.10.2026).

[8] OWASP Foundation. Authentication Cheat Sheet. URL: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html (дата обращения: 29.10.2026).

[9] OWASP Foundation. Session Management Cheat Sheet. URL: https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html (дата обращения: 29.10.2026).

[10] OWASP Foundation. Forgot Password Cheat Sheet. URL: https://cheatsheetseries.owasp.org/cheatsheets/Forgot_Password_Cheat_Sheet.html (дата обращения: 29.10.2026).

[11] National Institute of Standards and Technology. Digital Identity Guidelines: Authentication and Authenticator Management. NIST SP 800-63B-4. July 2025. URL: https://pages.nist.gov/800-63-4/sp800-63b.html (дата обращения: 29.10.2026).